Laboratório 06

Primeiro Contacto com Redes Definidas por Software

Infraestruturas de Computação na Nuvem | Aula 06

Objetivo e enquadramento

A Aula 06 apresenta, no plano teórico, a separação entre o plano de controlo e o plano de dados, o protocolo OpenFlow, as tabelas de fluxos e as redes overlay. Este laboratório torna esses conceitos observáveis numa rede virtual construída na máquina do próprio aluno, com o Mininet, o Open vSwitch e o controller Ryu.

O trabalho não requer acesso ao servidor da unidade curricular nem a qualquer serviço de nuvem. Destina-se a ser realizado durante a aula, ao ritmo de cada aluno, e **não constitui trabalho a entregar**. Todo o trabalho decorre numa máquina virtual Ubuntu, criada e apagada com um único comando, de modo que o computador do aluno permanece intacto.

Nota

O Mininet e o Open vSwitch exigem um núcleo Linux. Por esse motivo o laboratório utiliza uma máquina virtual criada com o `multipass`, que funciona da mesma forma em Windows, macOS e Linux. Quem utilizar Linux pode executar os passos diretamente no seu sistema, a partir do passo 1.3.

Resultados de aprendizagem

No final deste laboratório, o aluno deve ser capaz de:

- Criar uma topologia virtual com hosts e um switch Open vSwitch.
- Demonstrar que um switch sem controller nem regras não encaminha tráfego.
- Ligar um controller Ryu e observar o modelo reativo, com mensagens *packet-in*.

- Ler uma tabela de fluxos e identificar prioridade, contadores e *timeouts*.
- Instalar uma regra de bloqueio e explicar a razão pela qual prevalece sobre as restantes.
- Criar um túnel VXLAN entre dois namespaces e observar o encapsulamento em UDP.

Requisitos

- Computador com Windows, macOS ou Linux.
- Cerca de 2 GB de memória livres para a máquina virtual e 10 GB de espaço em disco.
- Ligação à Internet, necessária para obter a imagem e os pacotes.
- Permissão para instalar o `multipass` (Windows e macOS) ou o `snap` (Linux).
- Duração estimada: entre 75 e 100 minutos.

Como utilizar este guião

Cada passo apresenta, por esta ordem, o objetivo, o enquadramento, o fundamento teórico, os comandos a executar e o resultado esperado. Recomenda-se ler o passo completo antes de executar o primeiro comando. As caixas têm sempre o mesmo significado:

Comando a executar

```
Comando a executar no terminal.
```

Resultado esperado (os valores podem variar)

```
Resultado que deve ser apresentado.
```

Analisar

Momento de pausa para interpretar o resultado obtido, isoladamente ou em discussão com os colegas.

Erro possível e resolução

Erro que pode ocorrer e respetiva resolução.

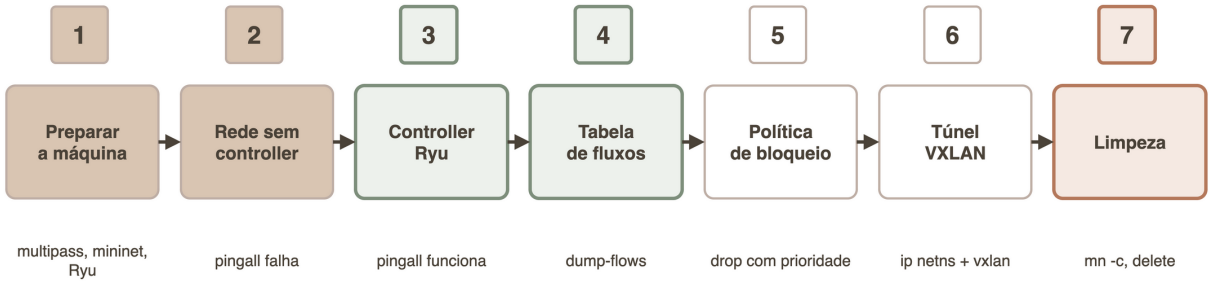
Nota

Informação complementar que facilita a execução.

Os comandos podem ser copiados e colados. Os resultados mostrados são exemplos: endereços, contadores, tempos e números de versão diferem de computador para computador. O prompt `mininet>` indica que o comando é escrito na consola do Mininet, e não no terminal da máquina virtual.

Percurso

A figura seguinte resume a sequência do guião. Os passos 3 e 4 constituem a observação central: um controller instala regras na tabela de fluxos de um switch, e essas regras são visíveis e mensuráveis.



Os passos 3 e 4 são a observação central: o controller instala regras na tabela

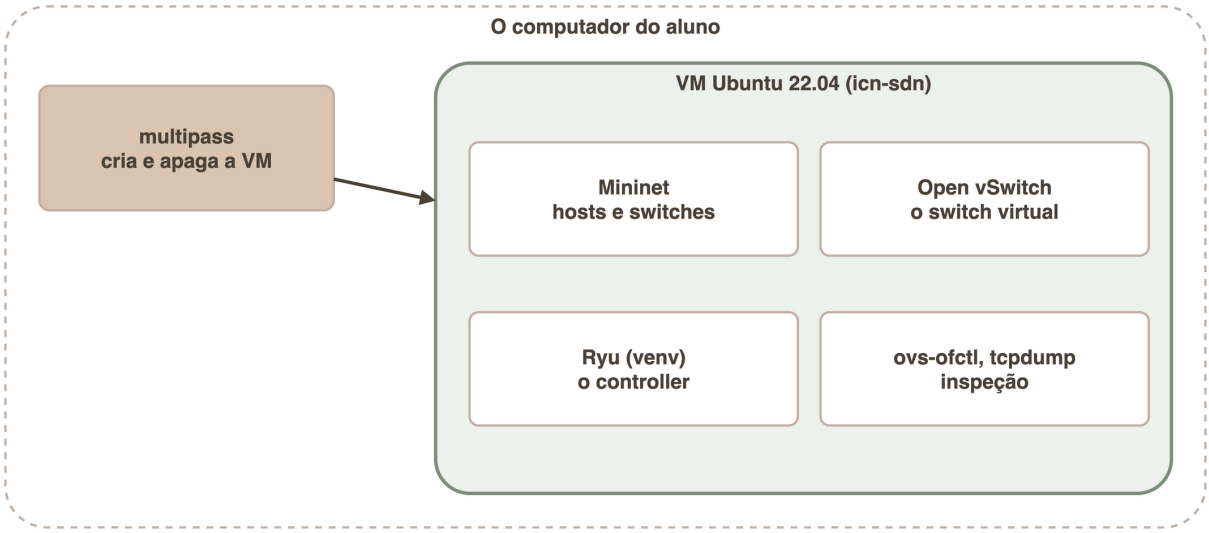
Como ler a figura

Os sete passos leem-se da esquerda para a direita. Os dois primeiros, a bege, preparam o ambiente e mostram o comportamento de uma rede sem controller. Os passos 3 e 4, a verde, são a parte central: liga-se o controller e lê-se a tabela de fluxos que ele construiu. Os passos 5 e 6, a branco, acrescentam uma política de bloqueio e um túnel VXLAN. O passo 7, em tom avermelhado, encerra o trabalho e liberta os recursos. Abaixo de cada caixa, uma legenda resume o que é feito nesse passo.

Passo 1 — Preparação do ambiente

Objetivo. Criar uma máquina virtual Ubuntu e instalar nela o Mininet, o Open vSwitch e o controller Ryu.

Enquadramento. O Mininet emula hosts e switches num único sistema Linux, recorrendo aos namespaces de rede estudados na Aula 04. Cada host do Mininet é, na prática, um processo com a sua própria pilha de rede.



Como ler a figura

A caixa exterior representa o computador do aluno. À esquerda está o `multipass`, o único programa instalado no computador, cuja função é criar e apagar a máquina virtual. A caixa verde é essa máquina virtual. Dentro dela estão as quatro peças do laboratório: o Mininet, que cria a rede emulada, o Open vSwitch, que é o switch usado nessa rede, o Ryu, que é o controller, e as ferramentas de inspeção. Tudo o que é instalado fica dentro da máquina virtual e desaparece quando esta é apagada.

Fundamento teórico

O Mininet não simula uma rede: emula-a. Cada host é um processo comum do Linux, colocado num namespace de rede próprio, com uma interface virtual ligada por um par *veth* ao switch. Os comandos que se executam num host correm, de facto, no núcleo real. Por isso o `ping` mede tráfego verdadeiro e o Open vSwitch utilizado é o mesmo que se encontra em hipervisores de produção.

O Ryu é um *framework* em Python para escrever controllers OpenFlow. Cada aplicação reage a eventos, como a ligação de um switch ou a chegada de um *packet-in*. Para ser executado de forma isolada, instala-se num ambiente virtual do Python, de modo a não

interferir com os pacotes do sistema.

A analisar neste passo: onde vive cada ferramenta. Apenas o `multipass` corre no computador do aluno, e tudo o resto corre dentro da máquina virtual.

1.1 Instalação do Multipass

Em macOS, com o Homebrew, executar o comando seguinte. Em Windows, utilizar o instalador disponível em https://multipass.run. Em Linux, o `multipass` instala-se com o `snap`.

Comando a executar

```
brew install --cask multipass
```

Comando a executar

```
sudo snap install multipass
```

O primeiro comando destina-se a macOS e o segundo a Linux. Confirmar a instalação:

Comando a executar

```
multipass version
```

Resultado esperado (os valores podem variar)

```
multipass   1.xx.x
```

Erro possível e resolução

command not found: fechar o terminal e abrir um novo. Em Windows, utilizar o PowerShell e confirmar que o instalador terminou sem erros.

1.2 Criação da máquina virtual

Comando a executar

```
multipass launch 22.04 --name icn-sdn --cpus 2 --memory 2G --disk 10G
multipass shell icn-sdn
```

Resultado esperado (os valores podem variar)

```
Launched: icn-sdn
ubuntu@icn-sdn:~$
```

A partir deste momento, todos os comandos são executados dentro da máquina virtual, no prompt `ubuntu@icn-sdn`. A primeira execução demora alguns minutos, por incluir a obtenção da imagem.

Erro possível e resolução

launch failed ou execução parada: o computador não dispõe de memória livre suficiente. Fechar outras aplicações e repetir. Se a máquina já existir, executar `multipass delete icn-sdn`, `multipass purge` e repetir.

1.3 Instalação do Mininet, do Open vSwitch e das ferramentas

Comando a executar

```
sudo apt update
sudo apt install -y mininet openvswitch-switch python3-venv tcpdump
```

Confirmar que o Open vSwitch e o Mininet respondem:

Comando a executar

```
sudo ovs-vsctl --version
sudo mn --version
```

Resultado esperado (os valores podem variar)

```
ovs-vsctl (Open vSwitch) 2.17.x
2.3.x
```

1.4 Instalação do Ryu

O Ryu é instalado num ambiente virtual. O pacote `eventlet` é fixado numa versão antiga, porque as versões recentes são incompatíveis com o Ryu.

Comando a executar

```
mkdir -p ~/icn-aula06
python3 -m venv ~/ryu-env
~/ryu-env/bin/pip install ryu "eventlet==0.30.1"
~/ryu-env/bin/ryu-manager --version
```

Resultado esperado (os valores podem variar)

```
ryu-manager 4.34
```

Erro possível e resolução

Falha na compilação do Ryu, com mensagens sobre `setuptools`: instalar primeiro uma versão anterior do `setuptools`, com `~/ryu-env/bin/pip install "setuptools==58.2.0"wheel`, e repetir a instalação do Ryu.
ImportError sobre `ALREADY_HANDLED` ao arrancar o `ryu-manager`: o `eventlet` instalado é demasiado recente. Executar `~/ryu-env/bin/pip install "eventlet==0.30.1"`.

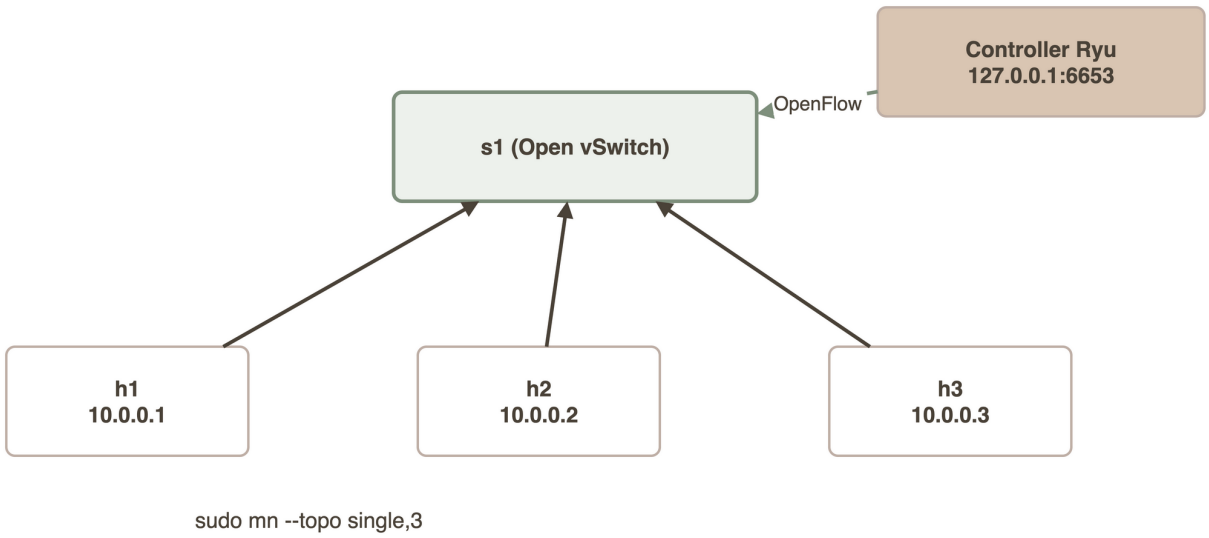
Verificação antes de avançar

Os comandos `sudo ovs-vsctl --version`, `sudo mn --version` e `~/ryu-env/bin/ryu-manager --version` executam sem erro.

Passo 2 — Uma rede sem controller

Objetivo. Criar uma topologia com três hosts e um switch, e demonstrar que, sem controller e sem regras, o switch não encaminha tráfego.

Enquadramento. Na Aula 06, o plano de dados foi apresentado como a parte que aplica regras já instaladas. Se não houver regras, o plano de dados não tem critério para decidir. Este passo mostra esse efeito.



Como ler a figura

No centro está o switch `s1`, a verde. Os três hosts, `h1`, `h2` e `h3`, ligam-se a ele, cada um com o seu endereço IP. O controller, em cima à direita, liga-se ao switch por OpenFlow, representado a tracejado porque ainda não existe neste passo. A topologia é sempre a mesma ao longo do guião; o que muda é quem decide as regras do switch.

Fundamento teórico

Um switch OpenFlow não aprende endereços por si. Cada pacote é comparado com as entradas da tabela de fluxos e, se nenhuma corresponder, o resultado depende do *fail-mode* do switch. No modo *secure*, que o Mininet utiliza, o switch não encaminha o que não conhece. No modo *standalone*, comportar-se-ia como um switch tradicional quando perdesse o controller.

A ação especial `NORMAL` pede ao switch que use a lógica clássica de camada 2: aprender endereços e inundar quando não conhece o destino. Instalar uma única regra com essa ação equivale a regressar ao switch tradicional, e é um exemplo extremo de modelo proativo: a regra foi instalada antes de haver tráfego, sem qualquer controller.

A analisar neste passo: a diferença entre ter um switch e ter um switch com regras. O hardware é o mesmo nas duas situações; o que muda é o conteúdo da tabela de fluxos.

2.1 Criação da topologia

Comando a executar

```
sudo mn --topo single,3 --mac --switch ovsk,protocols=OpenFlow13 --controller none
```

Resultado esperado (os valores podem variar)

```
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Starting CLI:
mininet>
```

O parâmetro `--mac` atribui endereços MAC simples (`00:00:00:00:00:01`, `...:02`, `...:03`), mais fáceis de reconhecer. O parâmetro `protocols=OpenFlow13` fixa a versão do protocolo.

2.2 Teste de conectividade

Comando a executar

```
mininet> pingall
```

Resultado esperado (os valores podem variar)

```
*** Ping: testing ping reachability
h1 -> X X
h2 -> X X
h3 -> X X
*** Results: 100% dropped (0/6 received)
```

Analisar

Os três hosts estão ligados ao mesmo switch e nenhum alcança os outros. O que falta ao switch? Onde é que, nesta rede, estaria a informação necessária para decidir?

2.3 A tabela de fluxos está vazia

O comando `sh` da consola do Mininet executa um comando no sistema da máquina virtual.

Comando a executar

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
```

Resultado esperado (os valores podem variar)

```
(sem entradas)
```

2.4 Instalação de uma regra única

Comando a executar

```
mininet> sh ovs-ofctl -O OpenFlow13 add-flow s1 actions=NORMAL
mininet> pingall
```

Resultado esperado (os valores podem variar)

```
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
```

Erro possível e resolução

ovs-ofctl: s1 is not a bridge or a socket: o Mininet não está a correr, ou o nome do switch está errado. Confirmar que o prompt `mininet>` está ativo e que o switch se chama `s1`.
Os pings continuam a falhar após o add-flow: confirmar com `dump-flows` que a regra foi instalada. Se a lista estiver vazia, repetir o comando e verificar a versão do protocolo

(`-O OpenFlow13`).

Verificação antes de avançar

O `pingall` falhou sem regras e funcionou depois de instalada a regra `actions=NORMAL`.

Sair do Mininet e limpar o ambiente antes do passo seguinte:

Comando a executar

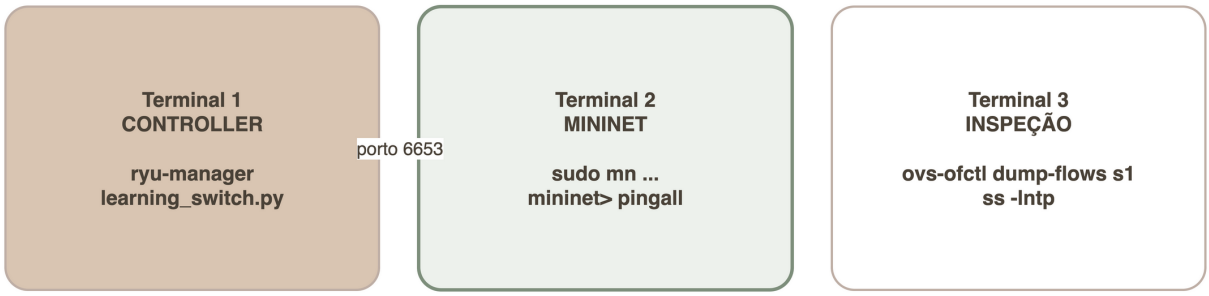
```
mininet> exit
sudo mn -c
```

Passo 3 — Um controller reativo com o Ryu

Objetivo. Escrever um controller que aprende a localização dos hosts, ligá-lo ao switch e observar a conectividade.

Enquadramento. Este passo concretiza o modelo reativo da Aula 06. O switch envia ao controller o primeiro pacote de cada fluxo, o controller decide e instala uma regra, e os pacotes seguintes já não passam por ele.

São necessários três terminais, todos ligados à mesma máquina virtual. Em cada novo terminal, executar `multipass shell icn-sdn`.



Os três terminais ligam-se à mesma VM: multipass shell icn-sdn

Como ler a figura

Cada caixa é um terminal. O terminal 1, a bege, executa o controller e deve ficar sempre aberto. O terminal 2, a verde, executa o Mininet, onde se escrevem os pings. O terminal 3, a branco, serve apenas para inspecionar: confirmar portos e listar fluxos sem interferir com os outros dois. A seta entre os dois primeiros indica que o Mininet se liga ao controller pelo porto 6653.

Fundamento teórico

Uma aplicação Ryu regista funções que reagem a eventos OpenFlow. O evento `EventOFPSwitchFeatures` ocorre quando um switch se liga, e é o momento de instalar a regra de último recurso: prioridade 0, qualquer pacote, enviar ao controller. É a entrada *table-miss*. O evento `EventOFPPacketIn` ocorre sempre que o switch envia um pacote ao controller.

O *learning switch* é a lógica clássica de camada 2, aqui em software. Ao receber um pacote, regista que o endereço MAC de origem está atrás da porta de entrada. Se já conhece a porta do destino, instala uma regra e encaminha. Se não conhece, inunda o pacote por

todas as portas. É o mesmo algoritmo que um switch tradicional executa internamente, mas agora a correr fora do switch.

A analisar neste passo: onde está cada plano. O plano de controlo é o ficheiro Python, e o plano de dados é a tabela de fluxos do Open vSwitch.

3.1 Ficheiro do controller

No terminal 1, criar o ficheiro `learning_switch.py` na pasta `icn-aula06` com o conteúdo seguinte, utilizando o `nano` (`nano ~/icn-aula06/learning_switch.py`).

Conteúdo do ficheiro `learning_switch.py`

```
from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import CONFIG_DISPATCHER, MAIN_DISPATCHER, set_ev_cls
from ryu.ofproto import ofproto_v1_3
from ryu.lib.packet import packet, ethernet


class LearningSwitch(app_manager.RyuApp):
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.mac_to_port = {}

    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
    def switch_features(self, ev):
        dp = ev.msg.datapath
        ofp, parser = dp.ofproto, dp.ofproto_parser
        actions = [parser.OFPActionOutput(ofp.OFPP_CONTROLLER, ofp.
    OFPCML_NO_BUFFER)]
        self.add_flow(dp, 0, parser.OFPMatch(), actions)

    def add_flow(self, dp, priority, match, actions, idle=0):
        ofp, parser = dp.ofproto, dp.ofproto_parser
        inst = [parser.OFPInstructionActions(ofp.OFPIT_APPLY_ACTIONS, actions)]
        dp.send_msg(parser.OFPFlowMod(datapath=dp, priority=priority, match=match,
                                      instructions=inst, idle_timeout=idle))

    @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def packet_in(self, ev):
        msg = ev.msg
        dp = msg.datapath
```

```
        ofp, parser = dp.ofproto, dp.ofproto_parser
        in_port = msg.match["in_port"]
        eth = packet.Packet(msg.data).get_protocols(ethernet.ethernet)[0]

        table = self.mac_to_port.setdefault(dp.id, {})
        table[eth.src] = in_port
        out_port = table.get(eth.dst, ofp.OFPP_FLOOD)
        actions = [parser.OFPActionOutput(out_port)]

        if out_port != ofp.OFPP_FLOOD:
            match = parser.OFPMatch(in_port=in_port, eth_dst=eth.dst)
            self.add_flow(dp, 1, match, actions, idle=60)

        data = msg.data if msg.buffer_id == ofp.OFP_NO_BUFFER else None
        dp.send_msg(parser.OFPPacketOut(datapath=dp, buffer_id=msg.buffer_id,
                                        in_port=in_port, actions=actions, data=
    data))
```

Erro possível e resolução

O Python distingue maiúsculas de minúsculas e depende da indentação, feita com espaços. Se o ficheiro for gravado com tabulações, o Ryu indica `IndentationError`. Rever o ficheiro e substituir as tabulações por quatro espaços.

3.2 Arranque do controller

Ainda no terminal 1:

Comando a executar

```
~/ryu-env/bin/ryu-manager ~/icn-aula06/learning_switch.py
```

Resultado esperado (os valores podem variar)

```
loading app /home/ubuntu/icn-aula06/learning_switch.py
loading app ryu.controller.ofp_handler
instantiating app /home/ubuntu/icn-aula06/learning_switch.py of LearningSwitch
instantiating app ryu.controller.ofp_handler of OFPHandler
```

O terminal fica ocupado: é o comportamento esperado. Deve permanecer aberto até ao fim do passo 5.

3.3 Confirmação do porto

No terminal 3, confirmar que o controller está à escuta:

Comando a executar

```
sudo ss -lntp | grep 6653
```

Resultado esperado (os valores podem variar)

```
LISTEN 0  128  0.0.0.0:6653  0.0.0.0:*  users:(("ryu-manager",pid=3120,fd=6))
```

Erro possível e resolução

Se não aparecer nenhuma linha, o controller não está a correr, ou usa outro porto. Confirmar o terminal 1. Se o porto indicado for o 6633, que é o valor histórico, usar esse valor no comando seguinte.

3.4 Ligação do Mininet ao controller

No terminal 2:

Comando a executar

```
sudo mn --topo single,3 --mac --switch ovsk,protocols=OpenFlow13 \
  --controller remote,ip=127.0.0.1,port=6653
```

Comando a executar

```
mininet> pingall
```

Resultado esperado (os valores podem variar)

```
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
```

Para confirmar que o switch está efetivamente ligado ao controller, no terminal 3:

Comando a executar

```
sudo ovs-vsctl show
```

Resultado esperado (os valores podem variar)

```
    Bridge s1
        Controller "tcp:127.0.0.1:6653"
            is_connected: true
        fail_mode: secure
        Port s1-eth1
        Port s1-eth2
        Port s1-eth3
```

Analisar

O resultado é o do passo 2 com a regra `NORMAL`, mas a origem é diferente. Desta vez, quem tomou as decisões foi um programa em Python que se pode ler e modificar. Onde está a linha `is_connected: true` e o que significaria `false`?

Erro possível e resolução

O pingall falha com o controller a correr: confirmar `is_connected: true` com `ovs-vsctl show`. Se for `false`, o porto do comando está errado ou o controller não arrancou. Sair do Mininet, executar `sudo mn -c` e repetir com o porto confirmado em 3.3.

Verificação antes de avançar

O `pingall` apresenta `0% dropped` e o `ovs-vsctl show` indica `is_connected: true`.

Passo 4 — A tabela de fluxos

Objetivo. Ler a tabela de fluxos construída pelo controller, identificar prioridade, contadores e *timeouts*, e observar a reconstrução reativa de uma regra.

Enquadramento. A tabela de fluxos é o ponto de contacto entre o plano de controlo e o plano de dados. Tudo o que o controller decidiu fica ali registado, com números que permitem verificar o que realmente aconteceu.

Prioridade	Match	Ação	Contadores	Timeout
100	ip, src 10.0.0.1, dst 10.0.0.3	drop	12 pkt	hard 60 s
10	eth_dst 00:..:02, in_port 1	output:2	840 pkt	idle 30 s
1	eth_dst 00:..:01, in_port 2	output:1	835 pkt	idle 30 s
0	qualquer pacote (table-miss)	enviar ao controller	7 pkt	sem

Vence a regra de maior prioridade que corresponda ao pacote

Como ler a figura

A figura é a mesma tabela de fluxos apresentada na aula. As colunas são as grandezas que se vão procurar no resultado de `dump-flows`: a prioridade, o *match*, a ação, os contadores e os *timeouts*. A linha de baixo, de prioridade 0, é a entrada *table-miss* que o controller instalou quando o switch se ligou. As restantes foram criadas à medida que o tráfego foi chegando.

Fundamento teórico

Cada entrada da tabela guarda os contadores `n_packets` e `n_bytes`, que o switch incrementa a cada pacote que corresponde. Não é necessário nenhum software adicional para medir o tráfego por fluxo. O campo `idle_timeout` indica ao switch que remova a entrada depois de um período sem pacotes, e o campo `duration` mostra há quanto tempo a entrada existe.

Quando uma entrada é removida, ou apagada à mão, o próximo pacote do fluxo volta a não encontrar regra e regressa ao controller. O mecanismo reativo repete-se: o controller decide de novo e reinstala a regra. É por isso que o primeiro `ping` depois de uma remoção demora mais do que os seguintes.

A analisar neste passo: a relação entre o que se vê na tabela e o tráfego que se gerou. Cada contador responde à pergunta: quantos pacotes usaram esta regra?

4.1 Leitura da tabela

No terminal 3:

Comando a executar

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
```

Resultado esperado (os valores podem variar)

```
cookie=0x0, duration=95.2s, table=0, n_packets=6, n_bytes=468, idle_timeout=60, priority
   =1,in_port="s1-eth1",dl_dst=00:00:00:00:00:02 actions=output:"s1-eth2"
cookie=0x0, duration=95.2s, table=0, n_packets=6, n_bytes=468, idle_timeout=60, priority
   =1,in_port="s1-eth2",dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
cookie=0x0, duration=103.9s, table=0, n_packets=18, n_bytes=1436, priority=0 actions=
   CONTROLLER:65535
```

Os nomes e os valores dependem do tráfego já gerado. Procurar nas entradas os campos `priority`, `n_packets`, `idle_timeout` e `duration`.

4.2 Os contadores aumentam

No terminal 2:

Comando a executar

```
mininet> h1 ping -c 5 h2
```

Voltar ao terminal 3 e repetir o `dump-flows`. O valor de `n_packets` das duas primeiras entradas deve ter aumentado em 5, uma por cada pedido e resposta.

4.3 Remoção das regras

Comando a executar

```
mininet> sh ovs-ofctl -O OpenFlow13 del-flows s1
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
```

Resultado esperado (os valores podem variar)

```
(sem entradas)
```

A remoção apagou também a entrada *table-miss*. Para que o switch volte a consultar o controller, é necessário reiniciar a ligação:

Comando a executar

```
mininet> sh ovs-vsctl del-controller s1
mininet> sh ovs-vsctl set-controller s1 tcp:127.0.0.1:6653
mininet> h1 ping -c 3 h2
```

Resultado esperado (os valores podem variar)

```
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=21.3 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.112 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.067 ms
```

Analisar

O primeiro pacote demorou muito mais do que os seguintes. Que percurso fez? Que mensagens trocaram o switch e o controller antes de ele ser entregue?

Erro possível e resolução

ping: connect: Network is unreachable ou `100% packet loss` depois da remoção: o switch ficou sem a entrada *table-miss* e sem controller. Repetir os dois comandos `ovs-vsctl` desta secção.

4.4 Expiração por inatividade

Aguardar mais de 60 segundos sem gerar tráfego e listar de novo a tabela:

Comando a executar

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
```

As entradas com `idle_timeout=60` desaparecem, e a entrada *table-miss* permanece, porque foi instalada sem *timeout*.

Verificação antes de avançar

Foi lida a tabela de fluxos, observado o aumento dos contadores, e verificado que a remoção de uma regra obriga o primeiro pacote a voltar ao controller.

Passo 5 — Uma política de bloqueio

Objetivo. Bloquear o tráfego entre `h1` e `h3`, mantendo a comunicação com `h2`, e observar o efeito da prioridade e do *timeout*.

Enquadramento. Uma política de rede traduz-se em regras com *match* mais específico e prioridade superior à das regras genéricas. É o mecanismo que, em ponto pequeno, sustenta a microssegmentação apresentada na aula.



Como ler a figura

O host `h1` envia tráfego para o switch. No switch coexistem duas regras que podem corresponder ao mesmo pacote: a de prioridade 100, que descarta o tráfego de `h1` para `h3`, e a de prioridade 1, que encaminha o tráfego em geral. A seta para `h2`, a cheio, representa o tráfego permitido. A seta para `h3`, a tracejado e a vermelho, representa o tráfego que a regra de maior prioridade descarta.

Fundamento teórico

Quando duas regras correspondem ao mesmo pacote, o switch aplica a de maior prioridade. A regra de encaminhamento do controller tem prioridade 1, pelo que uma regra de bloqueio com prioridade 100 prevalece sem ser necessário alterar o controller. A ação `drop` não é uma instrução: é a ausência de ações. Uma entrada sem ações descarta os pacotes que correspondem.

O *match* desta regra usa campos de camada 3, `ip`, `nw_src` e `nw_dst`, enquanto as regras do controller usam apenas camada 2. A regra não bloqueia os pedidos ARP, porque estes não são pacotes IP. O `h1` consegue descobrir o endereço MAC do `h3`, mas os pacotes IP seguintes são descartados.

O `hard_timeout` remove a regra ao fim de um tempo fixo, independentemente do tráfego.

Neste exercício serve para demonstrar uma política temporária que se levanta sozinha.

A analisar neste passo: a ordem de aplicação, que depende da prioridade e não da ordem de instalação das regras.

5.1 Instalação da regra

No terminal 2, com o Mininet ainda em execução:

Comando a executar

```
mininet> sh ovs-ofctl -O OpenFlow13 add-flow s1 "hard_timeout=60,priority=100,ip,
    nw_src=10.0.0.1,nw_dst=10.0.0.3,actions=drop"
```

5.2 Verificação

Comando a executar

```
mininet> h1 ping -c 3 h3
mininet> h1 ping -c 3 h2
```

Resultado esperado (os valores podem variar)

```
3 packets transmitted, 0 received, 100% packet loss, time 2049ms
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
```

Comando a executar

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
```

Resultado esperado (os valores podem variar)

```
 cookie=0x0, duration=14.2s, table=0, n_packets=3, n_bytes=294, hard_timeout=60, priority
    =100,ip,nw_src=10.0.0.1,nw_dst=10.0.0.3 actions=drop
 ...
```

O contador `n_packets` da regra de bloqueio indica quantos pacotes foram descartados.

5.3 Expiração da política

Aguardar 60 segundos desde a instalação e repetir:

Comando a executar

```
mininet> h1 ping -c 3 h3
```

Resultado esperado (os valores podem variar)

```
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
```

Analisar

Quem aplicou a política, o controller ou o switch? Em que local ficou escrita, e o que mudaria se o controller fosse reiniciado enquanto a regra existia?

Erro possível e resolução

O ping para h3 continua a funcionar: o `hard_timeout` pode já ter terminado, ou a regra não foi instalada. Confirmar com `dump-flows` e repetir o `add-flow`.

Verificação antes de avançar

O tráfego `h1` para `h3` foi descartado e o tráfego `h1` para `h2` manteve-se, e a política expirou ao fim de 60 segundos.

Desafio opcional: a política no controller

Mover a política para o plano de controlo. Em vez de instalar a regra com o `ovs-ofctl`, modificar o ficheiro `learning_switch.py`: no método `packet_in`, antes de calcular `out_port`, acrescentar uma verificação dos endereços MAC.

Conteúdo do ficheiro `learning_switch.py` (acrescento)

```
BLOQUEADOS = {("00:00:00:00:00:01", "00:00:00:00:00:03"),
              ("00:00:00:00:00:03", "00:00:00:00:00:01")}

# dentro de packet_in, antes de calcular out_port:
```

```
if (eth.src, eth.dst) in BLOQUEADOS:
    match = parser.OFPMatch(in_port=in_port, eth_src=eth.src, eth_dst=eth.dst)
    self.add_flow(dp, 100, match, [], idle=60)
    return
```

Parar o controller no terminal 1 com `Ctrl+C`, repetir o passo 3.2, sair do Mininet com `exit`, executar `sudo mn -c` e repetir o passo 3.4. Verificar com `h1 ping -c 3 h3` e `h1 ping -c 3 h2`.

Analisar

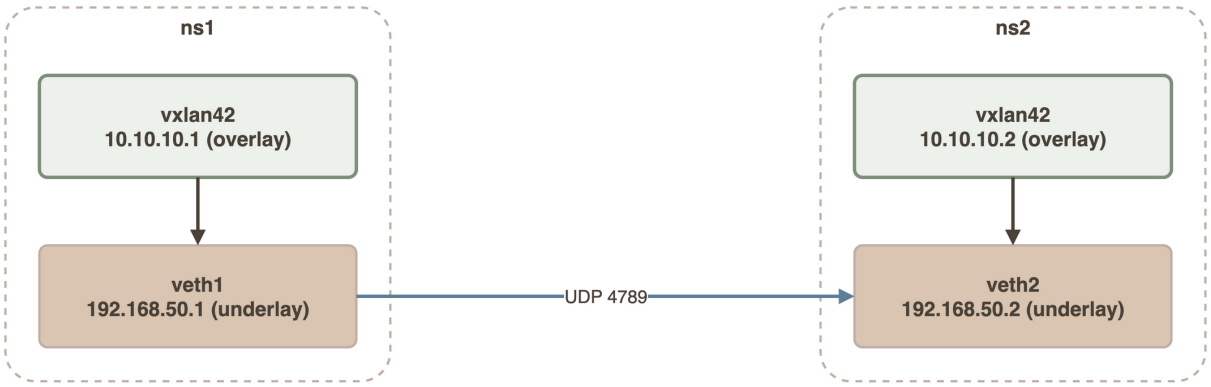
Qual a vantagem de a política viver no controller, em vez de ser instalada à mão em cada switch? E se a rede tivesse cinquenta switches?

Passo 6 — Um túnel VXLAN

Objetivo. Criar uma rede overlay entre dois namespaces, sobre uma rede subjacente, e observar o encapsulamento VXLAN em UDP.

Enquadramento. Os Pods do Kubernetes e as máquinas virtuais do Proxmox comunicam através de túneis como este. A diferença é que aqui o túnel é criado à mão, passo a passo, o que torna visível cada parte.

Este passo é independente do Mininet. Pode ser feito no terminal 3, ou em qualquer terminal da máquina virtual, depois de sair do Mininet com `exit` e de executar `sudo mn -c`.



O ping vai de 10.10.10.1 a 10.10.10.2; na veth só se vê UDP

Como ler a figura

Cada caixa tracejada é um namespace de rede, `ns1` à esquerda e `ns2` à direita. Em baixo, a bege, está o *underlay*: um par de interfaces `veth` com endereços da rede 192.168.50.0/24, ligadas entre si. Em cima, a verde, está o *overlay*: as interfaces `vxlan42`, com endereços da rede 10.10.10.0/24. Os pacotes do overlay descem para o underlay, atravessam-no em UDP no porto 4789 e voltam a subir no outro lado.

Fundamento teórico

Um namespace de rede é uma pilha de rede isolada, com as suas interfaces, tabelas de rotas e regras de firewall. É o mesmo mecanismo de isolamento dos containers da Aula 04. Um par *veth* é um cabo virtual: o que entra numa ponta sai na outra.

Uma interface VXLAN tem um identificador de rede, o VNI, e um destino remoto. Os pacotes que lhe são entregues são embrulhados em UDP e enviados ao destino remoto pela interface subjacente. O `ip` do Linux executa as funções de VTEP. O identificador 42 é arbitrário, mas tem de ser o mesmo nas duas extremidades.

O cabeçalho adicional ocupa 50 bytes. O Linux ajusta o MTU da interface overlay para 1450, de modo que um pacote encapsulado caiba no MTU de 1500 do underlay.

A analisar neste passo: a diferença entre o que os dois namespaces veem, pacotes IP normais entre 10.10.10.1 e 10.10.10.2, e o que passa na rede subjacente, UDP entre 192.168.50.1 e 192.168.50.2.

6.1 Criação do underlay

Comando a executar

```
sudo ip netns add ns1
sudo ip netns add ns2
sudo ip link add veth1 type veth peer name veth2
sudo ip link set veth1 netns ns1
sudo ip link set veth2 netns ns2
sudo ip netns exec ns1 ip addr add 192.168.50.1/24 dev veth1
sudo ip netns exec ns2 ip addr add 192.168.50.2/24 dev veth2
sudo ip netns exec ns1 ip link set veth1 up
sudo ip netns exec ns2 ip link set veth2 up
sudo ip netns exec ns1 ping -c 2 192.168.50.2
```

Resultado esperado (os valores podem variar)

```
64 bytes from 192.168.50.2: icmp_seq=1 ttl=64 time=0.071 ms
64 bytes from 192.168.50.2: icmp_seq=2 ttl=64 time=0.048 ms
```

6.2 Criação do overlay

Comando a executar

```
sudo ip netns exec ns1 ip link add vxlan42 type vxlan id 42 dev veth1 remote
    192.168.50.2 dstport 4789
sudo ip netns exec ns2 ip link add vxlan42 type vxlan id 42 dev veth2 remote
    192.168.50.1 dstport 4789
sudo ip netns exec ns1 ip addr add 10.10.10.1/24 dev vxlan42
sudo ip netns exec ns2 ip addr add 10.10.10.2/24 dev vxlan42
sudo ip netns exec ns1 ip link set vxlan42 up
sudo ip netns exec ns2 ip link set vxlan42 up
sudo ip netns exec ns1 ip -d link show vxlan42
```

Resultado esperado (os valores podem variar)

```
vxlan42: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 ...
    vxlan id 42 remote 192.168.50.2 dev veth1 srcport 0 0 dstport 4789 ...
```

6.3 Teste e captura

Abrir um segundo terminal da máquina virtual e iniciar a captura no underlay:

Comando a executar

```
sudo ip netns exec ns1 tcpdump -ni veth1 -c 4 udp port 4789
```

No primeiro terminal, gerar tráfego no overlay:

Comando a executar

```
sudo ip netns exec ns1 ping -c 2 10.10.10.2
```

Resultado esperado (os valores podem variar)

```
IP 192.168.50.1.43512 > 192.168.50.2.4789: VXLAN, flags [I] (0x08), vni 42
IP 10.10.10.1 > 10.10.10.2: ICMP echo request, id 11, seq 1, length 64
IP 192.168.50.2.43512 > 192.168.50.1.4789: VXLAN, flags [I] (0x08), vni 42
IP 10.10.10.2 > 10.10.10.1: ICMP echo reply, id 11, seq 1, length 64
```

Analisar

Cada pacote aparece em duas camadas: um IP externo, entre os endereços do underlay, e um IP interno, entre os endereços do overlay. Qual dos dois é visível para a rede física? E o que seria necessário alterar na rede física para criar uma segunda rede overlay com o VNI 43?

Erro possível e resolução

Cannot find device veth1 ou `RTNETLINK answers: File exists`: um passo anterior falhou ou foi repetido. Executar `sudo ip netns del ns1`, `sudo ip netns del ns2` e recomeçar em 6.1.
tcpdump: command not found: instalar com `sudo apt install -y tcpdump`.
O ping no overlay falha: confirmar com `ip -d link show vxlan42` que os dois lados

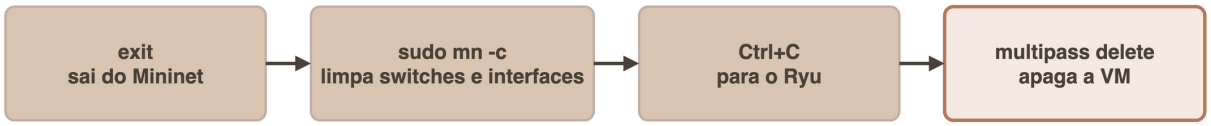
têm o mesmo VNI e que os endereços remotos estão trocados.

Verificação antes de avançar

O `tcpdump` mostra pacotes UDP no porto 4789 com `vni 42` a transportar os pedidos e respostas ICMP do overlay.

Passo 7 — Limpeza

Objetivo. Terminar o controller, apagar a topologia e a máquina virtual, libertando todos os recursos.



Os ficheiros learning_switch.py e comandos ficam no guião e permitem repetir tudo

Como ler a figura

A sequência da esquerda para a direita corresponde à ordem de execução. Primeiro sai-se do Mininet e limpam-se os switches e as interfaces que ele criou. Depois pára-se o controller. Por fim, a caixa avermelhada apaga a máquina virtual, e com ela tudo o que foi instalado.

7.1 Eliminação

Dentro da máquina virtual:

Comando a executar

```
mininet> exit
sudo mn -c
sudo ip netns del ns1
sudo ip netns del ns2
```

No terminal 1, parar o controller com `Ctrl+C`. Depois, no computador do aluno, fora da máquina virtual:

Comando a executar

```
exit
multipass delete icn-sdn
multipass purge
```

7.2 Verificação

Comando a executar

```
multipass list
```

Resultado esperado (os valores podem variar)

```
No instances found.
```

Nota

Para repetir o laboratório, bastam os comandos do passo 1 e o ficheiro `learning_switch.py`. Convém guardá-lo fora da máquina virtual, por exemplo com `multipass transfer icn-sdn:/home/ubuntu/icn-aula06/learning_switch.py .` antes de a apagar.

Resolução de problemas: consulta rápida

Sintoma	Resolução
`multipass: command not found`	Fechar e abrir o terminal. Se persistir, repetir a instalação.
`launch failed` ou VM parada	Libertar memória e repetir. Se existir, `multipass delete icn-sdn` e `multipass purge`.
Falha ao instalar o Ryu	Instalar `setuptools==58.2.0` e `wheel` no ambiente virtual e repetir.
`ALREADY_HANDLED` no `ryu-manager`	`pip install "eventlet==0.30.1"` no ambiente virtual do Ryu.
`Exception: Error creating interface pair`	Executar `sudo mn -c` e repetir o comando do Mininet.
`RTNETLINK: File exists` (Mininet)	Executar `sudo mn -c`. Um Mininet anterior ficou a meio.
`pingall` falha com controller ligado	`ovs-vsctl show` e ler `is_connected`. Confirmar o porto com `ss -lntp`.
`ovs-ofctl` sem resultados	Confirmar `-O OpenFlow13` e o nome do switch `s1`.
Módulo `openvswitch` em falta	`sudo apt install -y linux-modules-extra-$(uname -r)` ou usar `--switch ovs,datapath=user`.
`IndentationError` no Ryu	Substituir tabulações por quatro espaços no ficheiro.
Primeiro ping lento	Comportamento esperado do modelo reativo.
`Cannot find device veth1`	`sudo ip netns del ns1`, `sudo ip netns del ns2` e repetir o passo 6.

Comandos utilizados

Comando	Finalidade
`multipass launch 22.04 --name icn-sdn`	Criar a máquina virtual
`sudo mn --topo single,3 --mac`	Criar três hosts e um switch
`--controller none`	Arrancar o switch sem controller
`--controller remote,ip=...,port=...`	Ligar a um controller externo
`pingall`	Testar a conectividade entre todos os hosts
`ovs-ofctl -O OpenFlow13 dump-flows s1`	Listar a tabela de fluxos
`ovs-ofctl -O OpenFlow13 add-flow s1 ...`	Instalar uma regra
`ovs-ofctl -O OpenFlow13 del-flows s1`	Remover todas as regras
`ovs-vsctl show`	Mostrar bridges, portas e controller
`ryu-manager learning_switch.py`	Executar o controller
`ip netns add / exec`	Criar e usar um namespace de rede
`ip link add ...  type vxlan id 42`	Criar uma interface VXLAN
`tcpdump -ni veth1 udp port 4789`	Capturar tráfego VXLAN
`sudo mn -c`	Limpar switches e interfaces do Mininet
`multipass delete icn-sdn`	Apagar a máquina virtual

Questões de consolidação

As questões seguintes destinam-se a discussão e não têm de ser entregues.

1. Porque é que o `pingall` falhou no passo 2 e funcionou depois de uma única regra `NORMAL`? Onde estava o plano de controlo em cada caso?

2. O controller do passo 3 segue um modelo reativo ou proativo? Justificar com o que se observou no primeiro ping do passo 4.

3. Que informação usa o controller para escolher a porta de saída, e em que camada do modelo OSI se encontra?

4. Porque é que a regra de bloqueio prevaleceu sobre a regra de encaminhamento sem alterar

o controller?

5. Qual é o *underlay* e qual é o *overlay* no passo 6? Que parte do pacote é visível para a rede subjacente?

Síntese

O laboratório permitiu observar, na prática, as ideias centrais da Aula 06:

- Um switch sem regras não encaminha tráfego: o **plano de dados** depende do **plano de controlo**.
- O **controller** decide e o switch aplica. A tabela de fluxos é o ponto de contacto entre os dois.
- Cada entrada tem **prioridade**, **contadores** e **timeouts**, e é o que permite medir e governar o tráfego.
- No modelo **reativo**, o primeiro pacote de um fluxo vai ao controller, e os seguintes seguem a regra instalada.
- Uma **política** é uma regra mais específica e de maior prioridade, que se sobrepõe ao encaminhamento geral.
- Uma rede **overlay** embrulha o tráfego em UDP, e a rede física vê apenas o **underlay**.